

Explain map filter reduce



tell me about map, filter and reduce in details with example in javascript

Absolutely. `map()`, `filter()`, and `reduce()` are some of the **most important JavaScript array methods**, especially if you're learning modern JavaScript, React, or backend development.

They are all **higher-order functions**, meaning they can take another function (callback) as an argument.

1. `map()` in JavaScript

`map()` is used when you want to **transform every element of an array** and create a **new array**.

Syntax

```
JavaScript
array.map(callback);
```

More commonly:

```
JavaScript
const newArray = array.map((element, index, array) => {
  return something;
});
```

The callback can receive:

1. `element` → current element
2. `index` → current index
3. `array` → original array

Basic Example

```
JavaScript
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map((num) => {
  return num * 2;
});

console.log(doubled);
```

Output:

```
[2, 4, 6, 8, 10]
```

What happened?

`map()` went through every element:

```
1 → 1 * 2 → 2
2 → 2 * 2 → 4
3 → 3 * 2 → 6
4 → 4 * 2 → 8
5 → 5 * 2 → 10
```

The important thing is:

```
JavaScript
numbers
```

is **not** changed.

```
JavaScript
console.log(numbers);
```

Output:

```
[1, 2, 3, 4, 5]
```

2. `map()` with strings

```
JavaScript
const names = ["navi", "rahul", "aman"];

const upperNames = names.map((name) => {
  return name.toUpperCase();
});

console.log(upperNames);
```

Output:

```
["NAVI", "RAHUL", "AMAN"]
```

3. `map()` with index

```
JavaScript
const fruits = ["Apple", "Banana", "Mango"];

const result = fruits.map((fruit, index) => {
  return `${index}: ${fruit}`;
});

console.log(result);
```

Output:

```
[
  "0: Apple",
  "1: Banana",
  "2: Mango"
]
```

```
"2: Mango"
]
```

4. map() with objects

This is extremely important for React and real-world JavaScript.

```
JavaScript
const users = [
  { name: "Navi", age: 22 },
  { name: "Rahul", age: 25 },
  { name: "Aman", age: 20 }
];

const names = users.map((user) => {
  return user.name;
});

console.log(names);
```

Output:

```
["Navi", "Rahul", "Aman"]
```

You can also transform the objects.

```
JavaScript
const users = [
  { name: "Navi", age: 22 },
  { name: "Rahul", age: 25 }
];

const updatedUsers = users.map((user) => {
  return {
    name: user.name,
    age: user.age + 1
  };
});

console.log(updatedUsers);
```

Output:

```
JavaScript
[
  { name: "Navi", age: 23 },
  { name: "Rahul", age: 26 }
]
```

5. Important rule of map()

The callback should normally **return something**.

```
JavaScript
const numbers = [1, 2, 3];

const result = numbers.map(num => num * 2);

console.log(result);
```

Output:

```
[2, 4, 6]
```

But:

```
JavaScript
const result = numbers.map(num => {
  num * 2;
});

console.log(result);
```

Output:

```
[undefined, undefined, undefined]
```

Why?

Because with {}, you need an explicit **return**.

```
JavaScript
const result = numbers.map(num => {
  return num * 2;
});
```

6. filter() in JavaScript

filter() is used when you want to **select certain elements from an array based on a condition**.

It creates a **new array** containing only the elements for which the callback returns **true**.

Syntax

```
JavaScript
const result = array.filter((element) => {
  return condition;
});
```

Basic Example

Suppose we want only even numbers.

```
JavaScript
const numbers = [1, 2, 3, 4, 5, 6];

const evenNumbers = numbers.filter((num) => {
  return num % 2 === 0;
});

console.log(evenNumbers);
```

Output:

```
[2, 4, 6]
```

The process is:

```
1 → false → remove
2 → true  → keep
3 → false → remove
4 → true  → keep
5 → false → remove
6 → true  → keep
```

7. Filter numbers greater than 10

```
JavaScript
const numbers = [5, 12, 8, 20, 15, 3];

const result = numbers.filter((num) => {
  return num > 10;
});

console.log(result);
```

Output:

```
[12, 20, 15]
```

8. Filter strings

```
JavaScript
const names = ["Navi", "Aman", "Rahul", "Neha"];

const result = names.filter((name) => {
  return name.length > 4;
});

console.log(result);
```

Output:

```
["Rahul", "Neha"]
```

Actually, note that "Navi" has 4 characters and "Aman" has 4, so they are removed.

9. Filter objects

This is very common in applications.

```
JavaScript
const users = [
  { name: "Navi", age: 22 },
  { name: "Rahul", age: 17 },
  { name: "Aman", age: 25 },
  { name: "Neha", age: 16 }
];

const adults = users.filter((user) => {
```

```
return user.age >= 18;
});
console.log(adults);
```

Output:

```
JavaScript
[
  { name: "Navi", age: 22 },
  { name: "Aman", age: 25 }
]
```

10. map() VS filter()

This is very important.

map()

Transforms every element.

```
JavaScript
const numbers = [1, 2, 3];
const result = numbers.map(num => num * 10);
console.log(result);
```

Output:

```
[10, 20, 30]
```

Same number of elements.

filter()

Selects/removes elements.

```
JavaScript
const numbers = [1, 2, 3];
const result = numbers.filter(num => num > 1);
console.log(result);
```

Output:

```
[2, 3]
```

The number of elements can change.

Easy way to remember:

map → change
filter → select

11. reduce() in JavaScript

`reduce()` is probably the most powerful of the three.

It is used when you want to **combine all elements of an array into a single value**.

That single value could be:

- number
- string
- object
- array
- etc.

Syntax

```
JavaScript
const result = array.reduce((accumulator, currentValue) => {
  return newAccumulator;
}, initialValue);
```

The two most important things are:

```
JavaScript
accumulator
currentValue
```

12. Basic reduce() example

Let's calculate the sum.

```
JavaScript
const numbers = [10, 20, 30, 40];

const sum = numbers.reduce((total, num) => {
  return total + num;
}, 0);

console.log(sum);
```

Output:

```
100
```

Let's understand it step by step.

Initial value:

```
total = 0
```

Then:

```
0 + 10 = 10
10 + 20 = 30
30 + 30 = 60
60 + 40 = 100
```

Final result:

```
100
```

13. Understanding accumulator

This is the key to understanding `reduce()`.

```
JavaScript
const numbers = [1, 2, 3, 4];

const result = numbers.reduce((acc, current) => {
  console.log("acc:", acc);
  console.log("current:", current);

  return acc + current;
}, 0);
```

Conceptually:

```
Initial acc = 0

acc = 0, current = 1
return 1

acc = 1, current = 2
return 3

acc = 3, current = 3
return 6

acc = 6, current = 4
return 10
```

Final:

```
10
```

So:

The value returned by one iteration becomes the accumulator for the next iteration.

14. `reduce()` without initial value

You can technically write:

```
JavaScript
const numbers = [1, 2, 3, 4];

const sum = numbers.reduce((acc, num) => {
```

```
    return acc + num;
  });
  console.log(sum);
```

Output:

```
10
```

Here JavaScript uses the first array element as the initial accumulator.

Conceptually:

```
acc = 1
current = 2

acc = 3
current = 3

acc = 6
current = 4

acc = 10
```

But generally, providing an initial value is safer and clearer:

```
JavaScript
.reduce((acc, num) => acc + num, 0);
```

15. Find the maximum number using reduce()

```
JavaScript
const numbers = [10, 50, 20, 80, 30];

const max = numbers.reduce((max, num) => {
  return num > max ? num : max;
}, numbers[0]);

console.log(max);
```

Output:

```
80
```

16. Calculate total price

Very useful in e-commerce applications.

```
JavaScript
const products = [
  { name: "Laptop", price: 50000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 2000 }
];

const total = products.reduce((sum, product) => {
  return sum + product.price;
}, 0);
```

```
console.log(total);
```

Output:

```
53000
```

This pattern is extremely common:

```
JavaScript
array.reduce((acc, item) => acc + item.someProperty, 0);
```

17. Count elements using reduce()

Suppose:

```
JavaScript
const fruits = ["apple", "banana", "apple", "orange", "banana", "apple"];
```

We want to count how many times each fruit appears.

```
JavaScript
const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});

console.log(count);
```

Output:

```
JavaScript
{
  apple: 3,
  banana: 2,
  orange: 1
}
```

This is a very important real-world use of `reduce()`.

18. reduce() can create an object

For example:

```
JavaScript
const users = [
  { id: 1, name: "Navi" },
  { id: 2, name: "Rahul" },
  { id: 3, name: "Aman" }
];

const usersById = users.reduce((acc, user) => {
  acc[user.id] = user;
  return acc;
}, {});
```

```
}, {});  
console.log(usersById);
```

Result:

```
JavaScript  
{  
  1: { id: 1, name: "Navi" },  
  2: { id: 2, name: "Rahul" },  
  3: { id: 3, name: "Aman" }  
}
```

19. Combining filter() and map()

This is extremely common.

Suppose we have:

```
JavaScript  
const users = [  
  { name: "Navi", age: 22 },  
  { name: "Rahul", age: 17 },  
  { name: "Aman", age: 25 },  
  { name: "Neha", age: 16 }  
];
```

We want the names of adults.

First:

```
JavaScript  
const adults = users.filter(user => user.age >= 18);
```

Then:

```
JavaScript  
const names = adults.map(user => user.name);
```

Output:

```
["Navi", "Aman"]
```

You can combine them:

```
JavaScript  
const names = users  
  .filter(user => user.age >= 18)  
  .map(user => user.name);  
  
console.log(names);
```

Output:

```
["Navi", "Aman"]
```

20. Combining filter() + map() + reduce()

Now let's use all three.

Suppose:

```
JavaScript
const products = [
  { name: "Laptop", price: 50000, inStock: true },
  { name: "Mouse", price: 1000, inStock: false },
  { name: "Keyboard", price: 2000, inStock: true },
  { name: "Monitor", price: 15000, inStock: true }
];
```

We want the **total price of products that are in stock**.

Step 1: Filter

```
JavaScript
const availableProducts = products.filter(product => product.inStock);
```

Result:

```
JavaScript
[
  { name: "Laptop", price: 50000, inStock: true },
  { name: "Keyboard", price: 2000, inStock: true },
  { name: "Monitor", price: 15000, inStock: true }
]
```

Step 2: Reduce

```
JavaScript
const total = availableProducts.reduce((sum, product) => {
  return sum + product.price;
}, 0);
```

Result:

```
67000
```

Or everything together:

```
JavaScript
const total = products
  .filter(product => product.inStock)
  .reduce((sum, product) => sum + product.price, 0);

console.log(total);
```

Output:

```
67000
```

21. Using map() + filter() + reduce() together

Let's make it slightly more interesting.

JavaScript

```
const numbers = [1, 2, 3, 4, 5, 6];
```

Requirement:

1. Select even numbers
2. Multiply them by 10
3. Find their total

Step 1 — filter

JavaScript

```
const even = numbers.filter(num => num % 2 === 0);
```

Result:

```
[2, 4, 6]
```

Step 2 — map

JavaScript

```
const multiplied = even.map(num => num * 10);
```

Result:

```
[20, 40, 60]
```

Step 3 — reduce

JavaScript

```
const total = multiplied.reduce((sum, num) => sum + num, 0);
```

Result:

```
120
```

Everything together:

JavaScript

```
const total = numbers
  .filter(num => num % 2 === 0)
  .map(num => num * 10)
  .reduce((sum, num) => sum + num, 0);

console.log(total);
```

Output:

120

22. map() VS filter() VS reduce()

Method	Purpose	Returns	Changes array size?
map()	Transform elements	New array	 No
filter()	Select elements	New array	 Yes
reduce()	Combine elements	Single value	N/A

Think of them like this:

```
map()
↓
[1, 2, 3, 4]
↓
[2, 4, 6, 8]
TRANSFORM
```

```
filter()
↓
[1, 2, 3, 4]
↓
[2, 4]
SELECT
```

```
reduce()
↓
[1, 2, 3, 4]
↓
10
COMBINE
```

23. A very important interview concept

Suppose you are asked:

"How do you get the names of users whose age is greater than 18?"

You could write:

```
JavaScript
const users = [
  { name: "Navi", age: 22 },
  { name: "Rahul", age: 17 },
  { name: "Aman", age: 25 }
];

const result = users
  .filter(user => user.age > 18)
  .map(user => user.name);

console.log(result);
```

Output:

```
["Navi", "Aman"]
```

This demonstrates that you understand both:

```
filter → select the required users
map    → extract/transform their names
```

24. Another real-world example: Shopping cart

Imagine your shopping cart:

```
JavaScript
const cart = [
  { name: "Laptop", price: 50000, quantity: 1 },
  { name: "Mouse", price: 1000, quantity: 2 },
  { name: "Keyboard", price: 2000, quantity: 1 }
];
```

Calculate the total:

```
JavaScript
const total = cart.reduce((sum, item) => {
  return sum + item.price * item.quantity;
}, 0);

console.log(total);
```

Output:

```
54000
```

Because:

```
Laptop   → 50000 × 1 = 50000
Mouse    → 1000  × 2 = 2000
Keyboard → 2000  × 1 = 2000
          -----
          54000
```

25. Common mistakes

Mistake 1 — Using `map()` when you need filtering

Bad:

```
JavaScript
const result = numbers.map(num => {
  if (num > 10) {
    return num;
  }
});
```

This can produce:

```
[undefined, 20, undefined, 30]
```

Use:

```
JavaScript
const result = numbers.filter(num => num > 10);
```

Mistake 2 — Forgetting return

```
JavaScript
const result = numbers.map(num => {
  num * 2;
});
```

Wrong.

Use:

```
JavaScript
const result = numbers.map(num => {
  return num * 2;
});
```

Or:

```
JavaScript
const result = numbers.map(num => num * 2);
```

Mistake 3 — Wrong initial value in reduce()

For a sum:

```
JavaScript
.reduce((sum, num) => sum + num, 0);
```

For an object:

```
JavaScript
.reduce((acc, item) => {
  // ...
  return acc;
}, {});
```

For an array:

```
JavaScript
.reduce((acc, item) => {
  acc.push(item);
  return acc;
}, []);
```

The initial value should match the type of result you're building.

26. Quick mental model

Whenever you see an array problem, ask:

"Do I want to change every item?"

Use:

```
JavaScript
map()
```

"Do I want only some items?"

Use:

```
JavaScript
filter()
```

"Do I want one final result?"

Use:

```
JavaScript
reduce()
```

For example:

```
Array
├── Transform every item → map()
├── Select some items → filter()
└── Combine into one value → reduce()
```

27. One final example

```
JavaScript
const employees = [
  { name: "Navi", salary: 30000, active: true },
  { name: "Rahul", salary: 25000, active: false },
  { name: "Aman", salary: 40000, active: true },
  { name: "Neha", salary: 35000, active: true }
];
```

Get active employees:

```
JavaScript
const activeEmployees = employees.filter(employee => employee.active);
```

Get their names:

JavaScript

```
const names = activeEmployees.map(employee => employee.name);
```

Calculate their total salary:

JavaScript

```
const totalSalary = activeEmployees.reduce(
  (total, employee) => total + employee.salary,
  0
);
```

Or:

JavaScript

```
const totalSalary = employees
  .filter(employee => employee.active)
  .reduce((total, employee) => total + employee.salary, 0);
```

Result:

Active employees:

Navi
Aman
Neha

Total salary:

105000

Remember this

JavaScript

map()

"I have an array and I want a modified version of every item."

JavaScript

filter()

"I have an array and I only want the items that satisfy a condition."

JavaScript

reduce()

"I have an array and I want to turn all of it into one result."

Once you're comfortable with these three, the next important JavaScript array methods to learn are **forEach()**, **find()**, **findIndex()**, **some()**, **every()**, **sort()**, **includes()**, and **flatMap()**.

ChatGPT can make mistakes. Check important info.